UML documentation

1. Alert Generation System

The Alert Generation System evaluates incoming patient data against patients thresholds and routes alerts to the medical staff.

Design Rationale:

The diagram introduces two views of the alert subsystem. The main diagram (DiagramAlertGenerationSystem) shows the full runtime model; the supplementary diagram (DiagramPatientAlertGenerationSystem) shows the alert-generator that triggers or resolves an alert for given patient. .

- AlertGenerator is the core evaluator. It holds a reference to DataStorage (to pull current Patient records) and to AlertManager (to dispatch generated alerts).
- evaluateData(patient) iterates over a Patient's records, checks each against the patient's Threshold list, and calls the private triggerAlert() when a breach is detected. Keeping triggerAlert() private ensures that no external class can trigger alerts.
- Threshold encapsulates a single rule: a recordType, a numeric threshold, and an operator (e.g. '>'), plus the isBreeached(value) check. Threshold objects are owned by Patient, so that each patient has their own personalized thresholds. And they are accessible only through Patient; no subsystem can manipulate them directly.
- Patient.checkThreshold(recordType) provides a boolean check that AlertGenerator can call before pulling full records, reducing unnecessary storage queries under non-alert conditions.
- Alert is a simple value object: patientId, condition and timestamp. It is produced by AlertGenerator and dispatched by AlertManager.
- AlertManager owns a list of StaffMember objects and dispatches each alert to the correct staff via dispatchAlert(alert, staffId).
- The PatientDataGenerator interface and OutputStrategy interface (DiagramPatientAlertGenerationSystem) model the signal-generator boundary. AlertGenerator implements PatientDataGenerator, and the output side uses a OutputStrategy (console, socket, file) allowing the alert delivery channel to change without modifying the generator logic.

2. Data Storage System

The Data Storage System is responsible for storing incoming data. Medical staff with access to the data can retrieve, analyze, and delete, with regard to the deleting policies, the stored data.

Design Rationale

In the design data flows in through DataStorage and out through DataRetriever, with access to the data always checked (staffMember).

- DataStorage is the central store. It holds a Map<Integer, Patient> keyed by patient ID. It delegates actual data input to a DataReader interface, so it does not know how the data arrives.
- All read and delete operations go through DataRetriever, never directly through DataStorage.
- DataRetriever wraps DataStorage and enforces access control. Every retrieval or deletion method requires a StaffMember argument, and the hasAccess check on StaffMember ensures that only authorised personnel can read patient records. Every access is appended to an AccessLog (via logAccess).
- AccessLogEntry captures staffId, patientId, timestamp, and action, supporting review of actions of staffMebers.
- deleteAfter(timestamp, staffMember) implements a data deletion policy in which old records are eligible for removal. The staffMember parameter ensures deletions are always authorized and logged.
- StaffMember holds an accessiblePatients list, which is unique to each staff member. This insures that staff has access only to assigned patients.

3. Patient Identification System

The Patient Identification System is responsible for matching incoming data to the correct hospital patient, managing the integrity of those matches.

<u>Design Rationale</u>

- PatientIdentifier serves as the entry point for all incoming data streams. It receives raw measurement data (patient ID, value, record type, timestamp) and delegates matching to the IdentityManager. By keeping this class focused on routing incoming data, changes to how data arrives do not affect matching logic.
- IdentityManager owns the list of known HospitalPatient objects and performs the actual matching logic (matchHPatient). It also handles anomalies—if no match is found, it throws a PatientNotFoundException and logs the mismatch via logMismatches and flagMismatches. It also allows to create a new HospitalPatient form the inputed Patients personal data.
- HospitalPatient data (name, dateOfBirth, doctorId) is private and accessed only through PatientIdentifier's public interface, protecting patients personally information.
- HospitalPatient wraps the hospital record and exposes lifecycle methods (setAdmitted, setDischarged). It acts as a wrapper around the leaner Patient class, which holds list of PatientRecodrs (recorType, measurmentValue, timestamp, version). The patients records can be directly added and accessed only through Patient.
- PatientNotFoundException is modelled as an exception class thrown by IdentityManager, making error paths explicit in the design rather than hidden in prose.

4. Data Access Layer

The Data Access Layer is the boundary between the external signal generator and the rest of the Cardiovascular Health Monitoring System. It abstracts the transport protocol (TCP, WebSocket, or file), parses raw payloads, and hands off structured PatientRecord objects through Patient to DataStorage.

Design Rationale

The central design decision is the DataReader interface, which defines a single contract, readData(dataStorage: DataStorage), that all concrete readers must fulfil. Thanks to this the rest of the system never needs to know which reader is in use.

- TCPDataReader holds a ServerSocket and a client Socket, managing the connection lifecycle. It has its own DataParser.
- WebSocketDataReader owns a WebSocketServer reference and its own DataParser instance.
- FileDataReader holds a filePath and its own DataParser.
- DataParser is owned by composition inside each concrete reader, because parsing strategy is intrinsic to the reader: a file reader parsing CSV uses different logic than a TCP reader parsing binary frames. Keeping DataParser as a composed object (rather than a shared singleton) prevents one reader's parse errors from affecting others.
- DataStorage and Patient appear in this diagram to show where the parsed data lands.
- PatientRecord is the uniform output type from DataParser.parse(), providing a clean handoff point.